

# A Generalisable Pipeline for Backporting ROCm to Obsolete AMD Hardware

Executive summary for technical mentorship

## Core thesis

Build a reusable, benchmarked recovery pipeline for dropped AMD GPU support in ROCm, so older hardware can be revived without rediscovering the same failure modes from scratch.

## Why this matters

### Problem in one sentence

ROCm is the only credible open-source alternative to CUDA for GPU compute, but AMD periodically drops support for older architectures.

Capable hardware becomes software-stranded rather than physically obsolete.

This is especially acute for cost-constrained users relying on Ryzen APUs and Vega-class GPUs.

The gap is not a hardware failure; it is a

### What the project targets

dropped GPU support  
fragmented community fixes  
missing reproduction steps  
unclear performance cost

# The gap

---

## Current reality

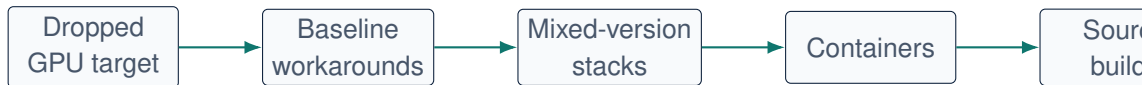
- scattered forum posts and issue threads
- ad hoc workarounds
- no single end-to-end reference
- little evidence about trade-offs

## What is missing

- reproducible steps
- benchmarked comparisons
- clear failure boundaries
- a path others can reuse

## Proposed approach: a decision tree for recovery

---



### Key idea

The project is a systematic pipeline, not a one-off workaround. Each strategy isolates a different failure layer, and each successful configuration is measured.

## Recovery strategies under test

| Strategy                    | Idea                                                      | What it tells us                                                                  |
|-----------------------------|-----------------------------------------------------------|-----------------------------------------------------------------------------------|
| Community base-line         | Environment-level spoofing and existing workarounds       | How far unsupported hardware runs without rebuilding the stack                    |
| Mixed-version stacks        | ROCm userspace and AMDGPU kernel version decoupling       | Whether failures come from driver ABI, userspace, or missing architecture support |
| Containerised vendor stacks | AMD Docker images across ROCm versions                    | Whether containers preserve the last usable stack or recover lost support         |
| Source builds               | Rebuild PyTorch and ROCm components with explicit targets | Whether compilation alone restores missing functionality                          |

## Scope: two representative targets, one reusable method

| Initial hardware targets                                                                                          | Reference environments from the draft                                                                                             |
|-------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| gfx90c — Ryzen 5 5650U / Cezanne APU / Vega 7<br>gfx902 — Ryzen 5 2200G / Raven Ridge / Vega 8                    | Fedora 43 on the 5650U system<br>RHEL 9.5 on the 2200G desktop system<br>ROCm 6.4.4 on the current host setup                     |
| Why these two                                                                                                     | Generalisation goal                                                                                                               |
| laptop and desktop form factors<br>same architectural family, different systems<br>cross-check for generalisation | The first traversal is about these two targets; the methodology is intended to transfer to other dropped AMD architectures later. |

## How success will be measured

### Evaluation at every stage

- direct HIP execution
- PyTorch runtime behavior
- performance relative to a working baseline
- reproducibility across systems and versions

### Initial observation

The current gfx900 spoof path for gfx90c reaches about 50% of AMD's published figures for the Vega 7.

### Comparison anchors

- latest stable ROCm for the target
- AMD's published figures
- the current workaround path

## What has already been established

---

### Post 1: context and motivation

Frames the ROCm ecosystem, the hardware target, and the case for running modern ML tooling on dropped GCN5 hardware.

### Post 2: installation and blocker

Shows native ROCm 6.4.4 installation via Fedora's DNF metapackage, successful HIP execution, and the central blocker: missing gfx90c kernels in bundled rocBLAS for PyTorch.

### Status signal

The hardware compute stack is functional; the unresolved issue is support continuity in ROCm userspace and packaged kernels.



## Execution path: from diagnosis to validation

---

### 1. Baseline

Test existing workarounds.

### 2. Decouple

Mix ROCm userspace and AMDGPU versions.

### 3. Containerise

Compare AMD Docker images across ROCm versions.

### 4. Rebuild

Compile PyTorch and needed ROCm components.

### 5. Validate

Re-run on gfx902.

### 6. Document

Publish steps, benchmarks, and failure modes.

# Why mentorship would matter

## What I am trying to validate

Whether dropped AMD GPU support can be recovered through spoofing, stack separation, containerisation, and selective source builds, with benchmarks at every stage.

## Focus areas

- tighten the technical methodology
- pressure-test the feasibility boundaries
- avoid dead ends that do not scale
- turn one hardware case into a reusable method

## Mentorship ask

I am asking for a GPU domain mentor with industry experience. I want practical guidance when I get stuck, and the FOSS United grants program requires a mentor.

## Closing line

The goal is not just to make one GPU work; it is to build a method others can use when ROCm drops the next architecture.